

UT 06.01 - El Lenguaje DQL. Agrupamientos.



Autor: Carlos Moreno Martínez.
cmorenomartinez@educa.madrid.org

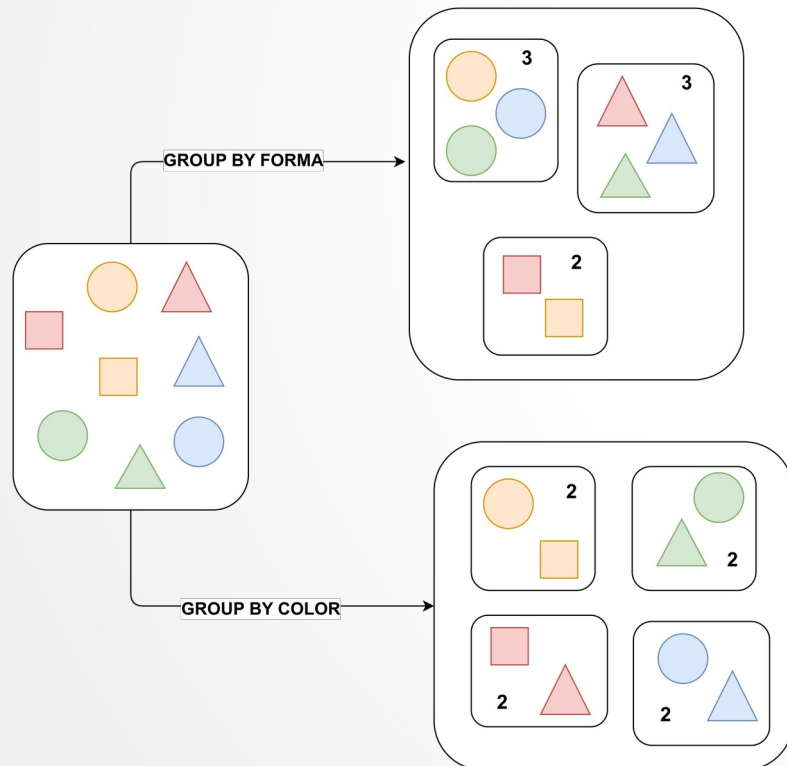


UT 06.01 - El Lenguaje DQL. Agrupamientos.

1.- La clausula de agrupamiento GROUP BY.

1.1.- Definición.

- Pero muchas veces es necesario conocer un determinado dato desglosado por un criterio. Por ejemplo, nos interesa conocer el salario medio de los empleados de cada uno de los departamentos.
- El algoritmo de cálculo es el siguiente:
 - El SGBD antes de realizar el cálculo organiza los registros agrupándolos por el criterio de agrupamiento.
 - Una vez agrupados, el SGBD realiza el calculo.
- Para indicar que se desea realizar una agrupación por un criterio se usa la cláusula **GROUP BY**.



- La sintaxis de la clausula básica es la siguiente:

```
SELECT
    campo_agrupamiento1,
    ...
    campo_agrupamientoN,

    func_agrupamiento1,
    ...
    func_agrupamientoN
FROM tabla
GROUP BY campo_agrupamiento1,..., campo_agrupamientoN;
```

UT 06.01 - El Lenguaje DQL. Agrupamientos.

1.- La clausula de agrupamiento GROUP BY.

1.2.- Campos de agrupamiento. Ejemplos.

- Son los campos según los cuales se realiza el agrupamiento de los datos de las filas.
- Este agrupamiento se realiza en el orden indicado en la clausula GROUP BY.

```
-- INDICAR CUANTOS EMPLEADOS HAY EN CADA DEPARTAMENTO.  
SELECT DEPTNO, COUNT(*) FROM EMP  
GROUP BY DEPTNO;
```

DEPTNO	COUNT (*)
30	6
20	5
10	3

```
-- PARA CADA UNO DE LOS DPTOS, INDICAR CUANTOS EMPLEADOS DE CADA OFICIO HAY.  
SELECT DEPTNO, JOB, COUNT(*) FROM EMP  
GROUP BY DEPTNO, JOB;
```

DEPTNO	JOB	COUNT (*)
20	CLERK	2
30	SALESMAN	4
20	MANAGER	1
30	CLERK	1
10	PRESIDENT	1
30	MANAGER	1
10	CLERK	1
10	MANAGER	1
20	ANALYST	2

UT 06.01 - El Lenguaje DQL. Agrupamientos.

1.- La clausula de agrupamiento GROUP BY.

1.2.- Campos de agrupamiento. Múltiples campos de agrupamiento.

- Podemos poner campos de la tablas y columnas calculadas por funciones de fila.

```
-- MOSTRAR UN LISTADO DE LOS EMPLEADOS DE CADA DPTO POR EL NUMERO Y LOS
-- MILES DE EUROS QUE GANAN.
SELECT
    DEPTNO,
    TRUNC(SAL/1000) MILES,
    COUNT(*)
FROM EMP
GROUP BY DEPTNO, TRUNC(SAL/1000);
```

DEPTNO	MILES	COUNT (*)
10	2	1
30	0	1
20	3	2
30	2	1
20	2	1
20	1	1
20	0	1
30	1	4
10	5	1
10	1	1

UT 06.01 - El Lenguaje DQL. Agrupamientos.

1.- La clausula de agrupamiento GROUP BY.

1.3.- Funciones de grupo. Funcionamiento.

Entre otras, tenemos las siguientes funciones de grupo:

Función	Significado
COUNT(expresión)	Cuenta las ocurrencias de expresión que no sean nulas.
SUM(expresión)	Suma los valores de la expresión.
AVG(expresión)	Calcula la media aritmética sobre la expresión indicada.
MIN(expresión)	Mínimo valor que toma la expresión indicada.
MAX(expresión)	Máximo valor que toma la expresión indicada.
STDDEV(expresión)	Calcula la desviación estándar.
VARIANCE(expresión)	Calcula la varianza.

- Las funciones de grupo actúan sobre los campos cuyo valor es distinto de NULL.
 - Si en una fila el campo donde se realiza la función de grupo es NULL simplemente se ignora a la hora de hacer el cálculo.
 - Si no deseamos ignorarles, deberemos utilizar funciones de nulos como COALESCE, NVL o NVL2 para manejar apropiadamente el valor nulo.

UT 06.01 - El Lenguaje DQL. Agrupamientos.

1.- La clausula de agrupamiento GROUP BY.

1.3.- Funciones de grupo. Ejemplos simples.

-- MOSTRAR LA SUMA DE LAS COMISIONES PAGADAS EN CADA DPTO (SIN NVL) .

```
SELECT
    DEPTNO,
    SUM(COMM)
FROM EMP
GROUP BY DEPTNO;
```

DEPTNO	SUM (COMM)
30	2200
20	
10	

-- MOSTRAR LA SUMA DE LAS COMISIONES PAGADAS EN CADA DPTO (CON NVL) .

```
SELECT
    DEPTNO,
    SUM(NVL (COMM, 0) )
FROM EMP
GROUP BY DEPTNO;
```

DEPTNO	SUM (NVL (COMM, 0))
30	2200
20	0
10	0

UT 06.01 - El Lenguaje DQL. Agrupamientos.

1.- La clausula de agrupamiento GROUP BY.

1.3.- Funciones de grupo. Ejemplos avanzados.

- Otra posibilidad de realizar cálculos de forma versátil es utilizar una combinación de la función suma y alguna función con especifique si el campo o campos cumple una condición.

```
-- CUANTOS PRESIDENT, CLERK y MANAGER HAY EN CADA DPTO.
```

```
SELECT
    DEPTNO,
    SUM(DECODE(JOB, 'PRESIDENT', 1, 0)) PRESIDENT,
    SUM(DECODE(JOB, 'CLERK', 1, 0)) CLERK,
    SUM(DECODE(JOB, 'MANAGER', 1, 0)) MANAGER
FROM EMP
GROUP BY DEPTNO;
```

DEPTNO	PRESIDENT	CLERK	MANAGER
30	0	1	1
20	0	2	1
10	1	1	1

```
-- MOSTRAR EN CUANTOS DPTOS HAY PERSONAL CADA UNO DE LOS OFICIOS.
```

```
SELECT
    JOB, COUNT(DISTINCT DEPTNO)
FROM EMP
GROUP BY JOB;
```

JOB	COUNT(DISTINCTDEPTNO)
CLERK	3
SALESMAN	1
PRESIDENT	1
MANAGER	3
ANALYST	1

UT 06.01 - El Lenguaje DQL. Agrupamientos.

2.- Filtrado de registros ANTES de agrupar. Clausula WHERE.

- Puede que sea necesario hacerlo con tan solo una parte de los datos que cumplan determinados criterios.
- Para ello, se añade la clausula WHERE que filtra dichos datos antes de la clausula GROUP BY de agrupamiento. La sintaxis queda de la forma siguiente:

```
SELECT  
    [campos_agrupamiento],  
    [funciones_agrupamiento]  
FROM tabla  
WHERE [condiciones]  
GROUP BY [campos_agrupamiento];
```

```
-- MOSTRAR EL NUMERO DE EMPLEADOS DE CADA OFICIO QUE ENTRO EN 1.981.  
SELECT JOB, COUNT(*)  
FROM EMP  
WHERE TO_CHAR(HIREDATE, 'YYYY') = '1981'  
GROUP BY JOB;
```

JOB	COUNT (*)
SALESMAN	4
CLERK	1
PRESIDENT	1
MANAGER	3
ANALYST	1

- Del mismo modo que existe la condición de búsqueda en filas individuales mediante la clausula WHERE, se dispone de un mecanismo para imponer una condición a los agrupamientos de filas. Esta es la clausula **HAVING**.
- La sintaxis es a siguiente:

```
SELECT
    [campos_agrupamiento],
    [funciones_agrupamiento]
FROM tabla
WHERE [condiciones_fila]
GROUP BY [campos_agrupamiento]
HAVING [condiciones_agrupamiento];
```

- No podemos imponer un HAVING si no realizamos antes un GROUP BY y ese el motivo por el cual en la sentencia vaya justo después una de la otra.
- Es importante entender que **la clausula HAVING se usa con funciones de grupo, no con campos de agrupación**. Si se desea aplicar una condición a un campo de agrupamiento se usa para ello la clausula WHERE.

UT 06.01 - El Lenguaje DQL. Agrupamientos.

3.- Filtrado de elementos en la agrupación. Clausula HAVING.

3.2.- Ejemplos.

```
-- MOSTRAR LOS DPTOS DONDE HAY MAS DE TRES EMPLEADOS.
```

```
SELECT
    DEPTNO,
    COUNT(*) NUM_EMPLEADOS
FROM EMP
GROUP BY DEPTNO
HAVING COUNT(*)>3;
```

DEPTNO	NUM_EMPLEADOS
30	6
20	5

```
-- MOSTRAR LOS OFICIOS DONDE LA MEDIA DE SU SALARIO SEA MAYOR DE 1,800.
```

```
SELECT
    JOB,
    TO_CHAR(ROUND(AVG(SAL), 2), '9G999D00') MEDIA_SAL
FROM EMP
GROUP BY JOB
HAVING AVG(SAL)>1800;
```

JOB	MEDIA_SAL
PRESIDENT	5.000,00
MANAGER	2.758,33
ANALYST	3.000,00

UT 06.01 - El Lenguaje DQL. Agrupamientos.

4.- Ordenando los resultados. Clausula ORDER BY.

- Puede que sea necesario hacerlo con tan solo una parte de los datos que cumplan determinados criterios. Para ello, se añade la clausula WHERE que filtra dichos datos antes de la clausula GROUP BY de agrupamiento. La sintaxis queda de la forma siguiente:

```
SELECT
    [campos_agrupamiento], [funciones_agrupamiento]
FROM tabla
WHERE [condiciones_fila]
GROUP BY [campos_agrupamiento]
HAVING [condiciones_agrupamiento]
ORDER BY [campos_agrupamiento | condiciones_agrupamiento];
```

```
-- MOSTRAR LA MEDIA DE LOS SALARIOS AGRUPADOS POR DPTO Y OFICIO. ORDENAR POR DPTO
-- DE FORMA ASCENDENTE Y OFICIO DESCENDIENTE.
```

```
SELECT
    DEPTNO, JOB, AVG(SAL) MEDIA
FROM EMP
GROUP BY DEPTNO, JOB
ORDER BY DEPTNO, JOB DESC;
```

DEPTNO	JOB	MEDIA
10	PRESIDENT	5000
10	MANAGER	2450
10	CLERK	1300
20	MANAGER	2975
20	CLERK	950
20	ANALYST	3000
30	SALESMAN	1400
30	MANAGER	2850
30	CLERK	950

UT 06.01 - El Lenguaje DQL. Agrupamientos.

5.- Extensiones adicionales a GROUP BY.

- Hay una serie de extensiones a la clausula GROUP BY que permiten extender la sentencia de tal forma que proporcionan potencialidades al operador.
- Con estas extensiones es posible obtener los datos deseados que necesitarían varias consultas mediante una única consulta.
- Vamos a ver tres tipos de consultas:
 - **GROUP BY ROLLUP.**
 - **CUBE.**
 - **GROUPING SETS.**



UT 06.01 - El Lenguaje DQL. Agrupamientos.

5.- Extensiones adicionales a GROUP BY.

5.1.- Clausula ROLL UP. Definición.



- ROLL UP crea **subresúmenes que van desde el nivel mas detallado hasta el resumen total.**
- Para obtener estos datos en SQL estándar, se debería realizar varias consultas para posteriormente unificarlos.
- La sintaxis básica de una sentencia con ROLL UP es la siguiente:

```
SELECT  
    [campos_agrupamiento],  
    [funciones_agrupamiento]  
FROM tabla  
GROUP BY ROLLUP (campos_agrupamiento);
```


UT 06.01 - El Lenguaje DQL. Agrupamientos.

5.- Extensiones adicionales a GROUP BY.

5.1.- Clausula ROLL UP. Ejemplo I.

```
SELECT
    deptno,
    job,
    SUM(SAL) "SUMSAL"
FROM emp
GROUP BY ROLLUP (deptno, job);
```

DEPTNO	JOB	SUMSAL
10	CLERK	1300
10	MANAGER	2450
10	PRESIDENT	5000
10		8750
20	CLERK	1900
20	ANALYST	6000
20	MANAGER	2975
20		10875
30	CLERK	950
30	MANAGER	2850
30	SALESMAN	5600
30		9400
		29025

13 rows selected.

En el ejemplo se puede percibir tres niveles de agrupamiento:

1. Suma de los salarios por oficio. Estas sumas se representan en amarillo.
2. Suma de los salarios totales de cada departamento. Estas sumas se representan en verde.
3. Suma de los salarios de todos los departamentos. Estas sumas se representan en azul.

UT 06.01 - El Lenguaje DQL. Agrupamientos.

5.- Extensiones adicionales a GROUP BY.

5.1.- Clausula ROLL UP. Ejemplo II.

Se desea realizar una estadística de las fechas de contratación de los empleados. Para ello, se desea que se desglose que número de empleados ha empezado a trabajar en que año y en que mes.

```
SELECT
  TO_CHAR(hiredate, 'YYYY') "AÑO",
  TO_CHAR(hiredate, 'MM') "NUMMES",
  COUNT(*) "NUMEP"
FROM EMP
GROUP BY ROLLUP TO_CHAR(hiredate, 'YYYY',
7          TO_CHAR(hiredate, 'MM'))
8  ORDER BY TO_CHAR(hiredate, 'YYYY'), TO_CHAR(hiredate, 'MM');
```

AÑO	NUMMES	NUMEP
-----	-----	-----
1980	12	1
1980		1
1981	2	2
1981	4	1
1981	5	1
1981	6	1
1981	9	2
1981	11	1
1981	12	2
1981		10
1982	1	1
1982		1
1987	4	1
1987	5	1
1987		2
		14

- **CUBE genera subtotales para todas las posibles combinaciones de agrupamientos especificados en la cláusula GROUP BY y una suma total.**

Si tiene n columnas o expresiones en la cláusula GROUP BY, habrá 2^n posibles combinaciones superagregadas.

- Matemáticamente, estas combinaciones forman un cubo de n dimensiones, de ahí el nombre del operador
- La sintaxis es la siguiente:

```
SELECT
    [campos_agrupamiento],
    [funciones_agrupamiento]
FROM tabla
GROUP BY CUBE (campos_agrupamiento);
```



UT 06.01 - El Lenguaje DQL. Agrupamientos.

5.- Extensiones adicionales a GROUP BY.

5.2.- Clausula CUBE. Ejemplo I.

```
SELECT
  deptno,
  job,
  sum(SAL) "SUMSAL"
FROM emp
GROUP BY CUBE (deptno, job)
ORDER BY deptno, job;
```

DEPTNO	JOB	SUMSAL
10	CLERK	1300
10	MANAGER	2450
10	PRESIDENT	5000
10		8750
20	ANALYST	6000
20	CLERK	1900
20	MANAGER	2975
20		10875
30	CLERK	950
30	MANAGER	2850
30	SALESMAN	5600
30		9400
	ANALYST	6000
	CLERK	4150
	MANAGER	8275
	PRESIDENT	5000
	SALESMAN	5600
		29025

18 rows selected.

Se pueden percibir los siguientes niveles de agrupamiento:

- Suma de los salarios por oficio y departamento. Estas sumas se representan en amarillo.
- Suma de los salarios totales de cada departamento. Estas sumas se representan en verde.
- Suma de los salario por oficio. Estas sumas se representan en naranja.
- Suma de los salarios de todos los departamentos. Estas sumas se representan en azul.

UT 06.01 - El Lenguaje DQL. Agrupamientos.

5.- Extensiones adicionales a GROUP BY.

5.2.- Clausula CUBE. Ejemplo II.

```
SELECT
  TO_CHAR(hiredate, 'YYYY') "AÑO",
  TO_CHAR(hiredate, 'MONTH') "MES",
  COUNT(*) "NUMEMP"
FROM emp
GROUP BY CUBE (TO_CHAR(hiredate, 'YYYY'), TO_CHAR(hiredate, 'MONTH'));
```

AÑO	MES	NUMEMP
		14
	ABRIL	2
	DICIEMBRE	3
	ENERO	1
	FEBRERO	2
	JUNIO	1
	MAYO	2
	NOVIEMBRE	1
	SEPTIEMBRE	2
1980		1
1980	DICIEMBRE	1
1981		10
1981	ABRIL	1
1981	DICIEMBRE	2
1981	FEBRERO	2
1981	JUNIO	1
1981	MAYO	1
1981	NOVIEMBRE	1
1981	SEPTIEMBRE	2
1982		1
1982	ENERO	1
1987		2
1987	ABRIL	1
1987	MAYO	1

24 rows selected.

UT 06.01 - El Lenguaje DQL. Agrupamientos.

5.- Extensiones adicionales a GROUP BY.

5.3.- Clausula GROUPING SETS. Definición.



- Mediante **GROUPING SETS** se puede definir varios agrupamientos en la misma consulta.

Oracle calcula todos los agrupamientos especificados en la cláusula y combina los resultados de agrupamientos individuales.

- La sintaxis es la siguiente:

```
SELECT  
    [campos_agrupamiento],  
    [funciones_agrupamiento]  
FROM tabla  
GROUPING SETS (agrupamientos);
```

- Tiene ventajas de eficiencia a la hora de la operatividad tales como que:
 - Sólo se requiere una transferencia sobre la tabla base.
 - Cuantos más elementos tenga **GROUPING SETS**, mayor será el beneficio en el rendimiento.

UT 06.01 - El Lenguaje DQL. Agrupamientos.

5.- Extensiones adicionales a GROUP BY.

5.1.- Clausula GROUPING SETS. Ejemplo I.

```
SELECT
  deptno,
  job,
  mgr,
  COUNT(*) "NUMEMP"
FROM emp
GROUP BY
  GROUPING SETS (
    (deptno, job),
    (job, mgr),
    (deptno, mgr)
  );
```

```
SELECT
  deptno,
  job,
  mgr,
  COUNT(*) "NUMEMP"
FROM emp
GROUP BY
  GROUPING SETS (
    (deptno, job),
    (job, mgr),
    (deptno, mgr)
  );
```

DEPTNO	JOB	MGR	NUMEMP
20	CLERK		2
30	SALESMAN		4
20	MANAGER		1
30	CLERK		1
10	PRESIDENT		1
30	MANAGER		1
10	CLERK		1
10	MANAGER		1
20	ANALYST		2
	CLERK	7902	1
	PRESIDENT		1
	CLERK	7698	1
	CLERK	7788	1
	CLERK	7782	1
	SALESMAN	7698	4
	MANAGER	7839	3
	ANALYST	7566	2
20		7839	1
10		7839	1
30		7698	5
20		7566	2
10		7782	1
20		7902	1
10			1
30		7839	1
20		7788	1

26 rows selected.

UT 06.01 - El Lenguaje DQL. Agrupamientos.

5.- Extensiones adicionales a GROUP BY.

5.1.- Clausula GROUPING SETS. Ejemplo II.

```
SELECT
  TO_CHAR(hiredate, 'YYYY') "AÑO",
  TO_CHAR(hiredate, 'MONTH') "MES",
  COUNT(*) "NUMEMP"
FROM emp
GROUP BY CUBE (TO_CHAR(hiredate, 'YYYY'), TO_CHAR(hiredate, 'MONTH'));
```

AÑO	MES	NUMEMP
		14
	ABRIL	2
	DICIEMBRE	3
	ENERO	1
	FEBRERO	2
	JUNIO	1
	MAYO	2
	NOVIEMBRE	1
	SEPTIEMBRE	2
1980		1
1980	DICIEMBRE	1
1981		10
1981	ABRIL	1
1981	DICIEMBRE	2
1981	FEBRERO	2
1981	JUNIO	1
1981	MAYO	1
1981	NOVIEMBRE	1
1981	SEPTIEMBRE	2
1982		1
1982	ENERO	1
1987		2
1987	ABRIL	1
1987	MAYO	1

24 rows selected.